[Bootcamp III] [Fases 1 a 3] [SDD]

System Design Document TaskAnalyzer

Especificação SDD consolidada — contrato de negócio, cenários de aceite e infraestrutura (CI/CD + Docker)

Autor	Alysson Kennedy de Oliveira Carvalho
RA	22450680 — Ciência da Computação
Repositório	github.com/AlyssonKennedy744/sdd-task-analyzer
Versão	1.0.0 (Fase 3)
Fonte da verdade	specs/task_analyzer_spec.md
Documento original (Fase 1)	Google Docs

Este arquivo é o **contrato executável** do módulo. Ele é a fonte da verdade para o assistente de IA (ver `CONTEXT_RULES.md`) e para o Test Harness (`tests/test_harness.py`).

---

1. Visão Geral e Contrato de Negócio

1.1 Propósito

O **TaskAnalyzer** é um módulo de análise de tarefas e produtividade. Ele recebe uma lista de tarefas (com datas, prazo, prioridade e status) e devolve, de forma determinística e sem persistência, um conjunto de métricas que apoiam a tomada de decisão de gestores e equipes.

Objetivos funcionais:

- Calcular o tempo médio de conclusão das tarefas (em minutos), geral e por prioridade.
- Calcular a taxa de atraso (percentual de tarefas concluídas após o prazo), geral e por prioridade.
- Gerar indicadores agrupados por prioridade (`baixa`, `media`, `alta`).
- Informar a quantidade de tarefas efetivamente consideradas na análise.
- Rejeitar, com exceções específicas e mensagens claras, entradas inválidas ou insuficientes.

1.2 Entradas — dataclass `Tarefa` (imutável, `frozen=True`)

Todos os campos de data/hora são `datetime` com fuso horário UTC (`tzinfo=timezone.utc`).

Campo	Tipo	Obrigatório	Descrição / Restrições
`id_tarefa`	`int`	Sim	Identificador único da tarefa (> 0). Duplicidade é erro.
`data_criacao`	`datetime`	Sim	Data/hora de criação (UTC).
`data_inicio`	`datetime | None`	Não	Início da execução. Obrigatória quando `status = concluida`.
`data_conclusao`	`datetime | None`	Não	Conclusão. Obrigatória quando `status = concluida`; deve ser >= `data_inicio`.
`prazo`	`datetime`	Sim	Data/hora limite para conclusão (UTC).
`prioridade`	`str` (enum)	Sim	`"baixa"`, `"media"`, `"alta"` (minúsculas, sem acento).
`status`	`str` (enum)	Sim	`"concluida"`, `"pendente"`, `"cancelada"`.

1.3 Saídas — dataclass `RelatorioMetricas`

Campo	Tipo	Descrição
`tempo_medio_conclusao_min`	`float`	Tempo médio de conclusão em minutos (geral), 2 casas.
`taxa_atraso_percentual`	`float`	% de tarefas concluídas após o prazo (geral), 0.00 a 100.00, 2 casas.
`quantidade_tarefas`	`int`	Total de tarefas consideradas (status `concluida` e datas válidas).

Campo	Tipo	Descrição
`indicadores_por_prioridade`	`dict[str, IndicadorPrioridade]`	Chaves fixas `baixa`, `media`, `alta`. Prioridade sem tarefas: `0.0, 0.0, 0`.

1.4 Regras de Negócio

- **RN-01.** Apenas tarefas com `status == "concluida"` entram no cálculo de tempo e de atraso.
- **RN-02.** Tempo de conclusão = `data_conclusao - data_inicio`, expresso em minutos (`float`).
- **RN-03.** Uma tarefa está em atraso quando `data_conclusao > prazo`. Conclusão exatamente no prazo não é atraso.
- **RN-04.** Tarefa concluída com datas inconsistentes (conclusão anterior à criação ou ao início, início anterior à criação, ou início/conclusão ausentes) é entrada inválida. Padrão: disparar `DatasInvalidasError`. Com `ignorar_inconsistentes=True`, a tarefa é descartada com aviso em log.
- **RN-05.** Divisão por zero: sem nenhuma tarefa concluída válida, `analisar_tarefas` dispara `SemTarefasConcluidasError`. Nunca retornar `NaN`, infinito ou zero silencioso.
- **RN-06.** Todo `datetime` deve ser timezone-aware em UTC. Valor naive dispara `DatasInvalidasError`.
- **RN-07.** Arredondamentos: 2 casas decimais para tempos e percentuais, usando `round(valor, 2)`.
- **RN-08.** Prioridade ou status fora dos valores permitidos dispara `ValorEnumInvalidoError`.
- **RN-09.** O módulo é puro: não lê nem escreve arquivos, não acessa rede e não mantém estado.
- **RN-10.** Todo descarte ou erro é registrado com `logging` (WARNING para descartes, ERROR antes de exceções).

> **Decisão de especificação (RN-04):** fail-fast como padrão (exceção) e modo tolerante via parâmetro explícito. Assim os dois comportamentos ficam testáveis e nenhum resultado incorreto é produzido de forma silenciosa.

1.5 Assinaturas Públicas e Exceções (contratuais)

```
# src/task_analyzer.py (Python 3.11+)
@dataclass(frozen=True)
class Tarefa:
    id_tarefa: int
    data_criacao: datetime
    prazo: datetime
    prioridade: str  # "baixa" | "media" | "alta"
    status: str      # "concluida" | "pendente" | "cancelada"
    data_inicio: datetime | None = None
    data_conclusao: datetime | None = None

@dataclass(frozen=True)
class IndicadorPrioridade:
    tempo_medio_conclusao_min: float
    taxa_atraso_percentual: float
    quantidade_tarefas: int

@dataclass(frozen=True)
class RelatorioMetricas:
    tempo_medio_conclusao_min: float
    taxa_atraso_percentual: float
    quantidade_tarefas: int
    indicadores_por_prioridade: dict[str, IndicadorPrioridade]

class TaskAnalyzerError(Exception): ...            # base de todas as exceções do módulo
class TaskValidationError(TaskAnalyzerError): ...  # base dos erros de entrada (Fase 2)
class DatasInvalidasError(TaskValidationError): ...
class ValorEnumInvalidoError(TaskValidationError): ...
class SemTarefasConcluidasError(TaskAnalyzerError): ...

def validar_tarefa(tarefa: Tarefa) -> None: ...
def calcular_tempo_conclusao_min(tarefa: Tarefa) -> float: ...
def esta_atrasada(tarefa: Tarefa) -> bool: ...
def analisar_tarefas(tarefas: list[Tarefa], ignorar_inconsistentes: bool = False) ->
RelatorioMetricas: ...
def analyze_tasks(tasks: list[dict[str, Any]]) -> dict[str, Any]: ...  # adendo Fase 2
```

Exceção	Quando é disparada	Mensagem esperada (exemplo)
`DatasInvalidasError`	Datetime naive; conclusão < criação; conclusão < início; início < criação; início/conclusão ausentes em tarefa concluída.	`Tarefa 7: data_conclusao (2026-09-01 08:00 UTC) é anterior a data_inicio (2026-09-01 09:00 UTC).`
`SemTarefasConcluidasError`	Lista vazia ou sem nenhuma tarefa concluída válida.	`Nenhuma tarefa concluída válida para calcular métricas (0 de 3 tarefas consideradas).`
`ValorEnumInvalidoError`	`prioridade` ou `status` fora dos valores permitidos.	`Tarefa 2: prioridade 'urgente' inválida. Valores aceitos: baixa, media, alta.`
`TaskValidationError`	Base das anteriores de entrada; também para `id_tarefa` <= 0, id duplicado ou campo obrigatório ausente em `analyze_tasks`.	`Tarefa 3: id_tarefa duplicado.`

1.6 Adendo Fase 2 — interface `analyze_tasks` (dict)

O enunciado da Fase 2 exige a função `analyze_tasks(tasks: list[dict]) -> dict` e a exceção `TaskValidationError`. Para **não violar PR-05/PR-10** (assinaturas e retorno de `analisar_tarefas` continuam intactos), ela foi adicionada como **fachada** sobre o mesmo núcleo de validação e cálculo:

- **Entrada:** lista de `dict` com as mesmas chaves da dataclass `Tarefa`. Datas podem ser `datetime` UTC ou string ISO 8601 com fuso (`"2026-09-01T08:00:00+00:00"` ou sufixo `"Z"`).
- **Validação:** qualquer entrada inválida dispara `TaskValidationError` (ou uma subclasse dela), inclusive `data_conclusao < data_criacao`.
- **Divisão por zero:** lista vazia ou sem tarefas concluídas **não** dispara exceção: médias e taxas retornam `0.0` (diferente de `analisar_tarefas`, que é fail-fast — RN-05).
- **Saída:**

Chave	Tipo	Descrição
`total_tarefas`	`int`	Quantidade de tarefas recebidas.
`total_concluidas`	`int`	Tarefas com status `concluida`.
`total_pendentes`	`int`	Tarefas com status `pendente`.
`total_canceladas`	`int`	Tarefas com status `cancelada`.
`tempo_medio_conclusao_min`	`float`	Média geral em minutos (RN-02, RN-07).
`tempo_medio_conclusao_horas`	`float`	Média geral em horas, `round(min / 60, 2)`.
`taxa_atraso_percentual`	`float`	% geral de concluídas após o prazo.
`metricas_por_prioridade`	`dict[str, dict]`	Para `baixa`, `media`, `alta`: `total_concluidas`, `tempo_medio_conclusao_horas`, `taxa_atraso_percentual`.

---

2. Cenários de Aceite (Dado–Quando–Então)

Não podem ser alterados, removidos ou adaptados pela IA (PR-02).

2.1 Conjunto de dados de referência (fixture)

Todas as datas são de 01/09/2026, em UTC. `data_criacao` de todas as tarefas: 07:00 UTC.

id	prioridade	status	data_inicio	data_conclusao	prazo	Tempo (min)	Atrasada?
1	alta	concluida	08:00	10:00	12:00	120	Não
2	alta	concluida	09:00	12:00	11:00	180	Sim
3	media	concluida	08:00	08:30	09:00	30	Não
4	baixa	concluida	08:00	09:00	08:30	60	Sim
5	media	pendente	—	—	18:00	ignorada	ignorada
6	alta	cancelada	07:00	—	10:00	ignorada	ignorada

2.2 Cenário 1 (Sucesso) — métricas corretas com tarefas válidas

Dado o conjunto de referência (6 tarefas, 4 concluídas com datas válidas), **Quando** `analisar_tarefas(tarefas)` for executado, **Então** deve retornar um `RelatorioMetricas` com exatamente:

Métrica	Geral	alta	media	baixa
`tempo_medio_conclusao_min`	97.50	150.00	30.00	60.00
`taxa_atraso_percentual`	50.00	50.00	0.00	100.00
`quantidade_tarefas`	4	2	1	1

Conferência: (120 + 180 + 30 + 60) / 4 = 97,5 min; atrasadas 2 de 4 = 50,00 %. Via `analyze_tasks`: `total_tarefas = 6`, `total_concluidas = 4`, `total_pendentes = 1`, `tempo_medio_conclusao_horas = 1.62` (97,5 / 60 = 1,625 → `round(x, 2)`), alta = 2.5 h, media = 0.5 h, baixa = 1.0 h.

2.3 Cenário 2 (Exceção/Erro)

2a — datas inconsistentes. Dado uma tarefa concluída cuja `data_conclusao` (08:00) é anterior à `data_inicio` (09:00), quando executado no modo padrão, então dispara `DatasInvalidasError` com o id da tarefa e as duas datas, sem retornar métricas parciais. Via `analyze_tasks`, `data_conclusao` anterior à `data_criacao` dispara `TaskValidationError`.

2b — divisão por zero. Dado uma lista vazia ou só com tarefas pendentes/canceladas, quando `analisar_tarefas` for executado, então dispara `SemTarefasConcluidasError` (nunca `ZeroDivisionError` nem `NaN`). Via `analyze_tasks`, o retorno é `0.0` para médias e taxas.

2.4 Cenários complementares

ID	Dado	Então
C3	Tarefa concluída exatamente no prazo.	Não é atraso (taxa 0.00 %).
C4	Datas inconsistentes e `ignorar_inconsistentes=True`.	Tarefa descartada, WARNING registrado, demais métricas calculadas.
C5	Prioridade `"urgente"`.	`ValorEnumInvalidoError` listando os valores aceitos.
C6	Datetime sem tzinfo (naive).	`DatasInvalidasError` informando que o valor deve ser UTC timezone-aware.
C7	Prioridade sem nenhuma tarefa concluída.	`indicadores_por_prioridade["baixa"] = (0.0, 0.0, 0)`, sem exceção.

2.5 Mapeamento para o Test Harness (`tests/test_harness.py`)

Cenário	Função de teste
Fixture 2.1	`tarefas_referencia()` / `tarefas_referencia_dict()`
Cenário 1	`test_metricas_corretas_com_tarefas_validas`, `test_analyze_tasks_cenario_1_sucesso`
Cenário 2a	`test_datas_inconsistentes_disparam_excecao`, `test_analyze_tasks_conclusao_anterior_a_criacao`

Cenário	Função de teste
Cenário 2b	`test_sem_tarefas_concluidas_dispara_excecao`, `test_analyze_tasks_casos_de_borda_retornam_zero`
C3	`test_conclusao_no_prazo_nao_e_atraso`
C4	`test_modo_tolerante_descarta_e_loga`
C5	`test_prioridade_invalida`
C6	`test_datetime_naive_e_rejeitado`
C7	`test_prioridade_sem_tarefas_retorna_zeros`

Critérios de aceite do harness: execução com `pytest -q` na raiz, 100 % dos testes passando antes de abrir Pull Request, testes determinísticos (sem rede, arquivos ou `datetime.now`).

---

3. Estrutura do Repositório

Estrutura final (Fase 3), autorizada pelo enunciado do Desafio Final e registrada aqui para manter a PR-03 válida:

```
sdd-task-analyzer/
├── .github/
│   └── workflows/
│       └── ci.yml                    # Motor CI/CD (GitHub Actions)
├── Dockerfile                        # Empacotamento (python:3.11-slim)
├── docker-compose.yml                # Orquestração da suíte isolada
├── .dockerignore
├── CONTEXT_RULES.md                  # Base de governança (Fase 1)
├── specs/
│   ├── system_design_document.pdf    # SDD consolidado (Fases 1 a 3)
│   ├── requirements.yaml             # Requisitos em formato estruturado
│   └── task_analyzer_spec.md         # Contrato executável (fonte da verdade)
├── tests/
│   └── test_harness.py               # Suíte de testes (Fase 2)
├── src/
│   └── task_analyzer.py              # Código gerado por IA e homologado (Fase 2)
├── .gitignore
├── requirements.txt
└── README.md
```

4. Versionamento (Git Flow simplificado)

- Branches: `main` (estável e homologada), `develop` (integração) e `feature/*`.
- Commits atômicos no padrão Conventional Commits (`feat:`, `fix:`, `test:`, `docs:`, `chore:`).
- Todo código chega à `develop` e à `main` exclusivamente via Pull Request.
- Cada PR descreve: cenário atendido, prompt utilizado, trechos gerados por IA e resultado do pytest.
- Tag `v0.1.0` ao final da Fase 2 e `v1.0.0` ao final da Fase 3.
- Fase 3: merge em `develop` e `main` só com o workflow `CI` verde (IN-01).

---

5. Adendo Fase 3 — CI/CD, Containerização e Governança

O contrato de negócio (seções 1 e 2) **não muda** na Fase 3. As alterações são apenas de infraestrutura: o mesmo código e os mesmos 13 testes passam a rodar de forma automática e isolada.

5.1 Integração Contínua (`.github/workflows/ci.yml`)

Item	Definição
Gatilhos	`push` e `pull_request` nas branches `main` e `develop` (e `workflow_dispatch`).
Ambiente	`ubuntu-latest` com Python 3.11 (`actions/setup-python`).
Preparação	`pip install -r requirements.txt` .
Execução	`pytest tests/test_harness.py -v` .
Job `docker`	Depende do job `test` : `docker build` + `docker compose run --rm tests` .
Critério	Build verde (13 passed) obrigatório antes do merge (IN-01).

5.2 Containerização (`Dockerfile` e `docker-compose.yml`)

Item	Definição
Imagem base	`python:3.11-slim` (leve e imutável).
Conteúdo	`requirements.txt` , `src/` e `tests/` (o `.dockerignore` exclui `.git` , specs e docs).
Usuário	`analyzer` (UID 1000), sem privilégios de root.
Comando padrão	`CMD ["pytest", "tests/test_harness.py", "-v"]` .
Orquestração	Serviço `tests` : `docker compose run --rm tests` , com `network_mode: none` e `read_only: true` (reforça a RN-09).

5.3 Requisitos estruturados

`specs/requirements.yaml` repete, em formato legível por máquina, as regras de negócio, as assinaturas públicas, os cenários de aceite com os valores esperados e os critérios de entrega. `specs/system_design_document.pdf` é a versão consolidada desta especificação para leitura e auditoria. Se houver divergência, este arquivo (`task_analyzer_spec.md`) prevalece.

5.4 Governança da IA na Fase 3

As regras IN-01 a IN-05 de `CONTEXT_RULES.md` valem para qualquer arquivo de infraestrutura gerado com IA: nada de `continue-on-error` , `|| true` ou testes pulados para forçar build verde, nenhum segredo no repositório e nenhuma imagem base diferente da especificada.